Experiment No. 6

Implementation and Comparative Analysis of
Image Segmentation Techniques using Python and OpenCV

Aim

To implement and evaluate various image segmentation techniques for partitioning digital images into meaningful regions, thereby facilitating object localization, scene understanding, and subsequent computer vision tasks.

Purpose

Image segmentation divides an image into multiple meaningful regions or objects based on pixel intensity, color, texture, or boundaries. Effective segmentation isolates the regions of interest (ROI), making tasks such as object detection, medical diagnosis, autonomous navigation, and industrial inspection more accurate and efficient.

In this experiment, Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering are implemented and their performance is compared.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, Matplotlib, and Scikit-learn
- Understanding of grayscale images, image histograms, and clustering concepts
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a real-world image (medical, satellite, or natural scene).
2. Convert the image to grayscale and reduce noise using Gaussian Blur.
3. Apply Global Thresholding to separate foreground from background.
4. Implement Otsu's Thresholding and compare it with global thresholding.
5. Apply Adaptive Thresholding to segment images with uneven illumination.
6. Implement Watershed Segmentation to separate overlapping or touching objects.
7. Apply K-Means Clustering for color-based segmentation.
8. Compare the outputs based on object boundaries, accuracy, and computational efficiency.
9. Visualize the results and identify the most suitable technique for the dataset.
10. Summarize the observations and discuss the role of segmentation in advanced computer vision.

Key Concepts and Formulas

- **Global threshold:** $g(x, y) = 255$ if $f(x, y) > T$, else 0, with one fixed T for the whole image.
- **Otsu's method:** chooses T that maximizes the between-class variance $\text{sigma}_B^2(T) = w_0\, w_1\, (\text{mu}_0 - \text{mu}_1)^2$ (equivalently minimizes within-class variance) using the histogram.
- **Adaptive threshold:** $T(x, y)$ = (mean or Gaussian-weighted mean of the neighborhood) - C, so each pixel has its own threshold.

- **Watershed:** treats the image as a topographic surface; markers are flooded until basins meet, and the meeting lines become object boundaries. The distance transform is used to find object centres as markers.
- **K-Means:** each pixel is a point (R, G, B); iteratively assign pixels to the nearest of K centroids and recompute centroids to minimize the sum of squared distances; replace each pixel by its centroid color.

Observations

- Global thresholding works only when the lighting is uniform and the histogram is clearly bimodal; with uneven illumination part of the background was classified as object.
- Otsu's method picks the threshold automatically from the histogram, but it is still a single global threshold, so it also fails under strong illumination gradients.
- Adaptive thresholding handles uneven illumination well because each pixel is compared with its local neighborhood. The block size must be larger than the objects, otherwise large uniform objects become hollow.
- Thresholding alone merges touching objects into one connected region. Watershed (with markers from the distance transform) separated them and gave the correct object count.
- K-Means groups pixels by color only, ignoring spatial position, so it works well for color-distinct regions but can break up or merge regions with similar color, and the result depends on K.
- Thresholding methods are the fastest (well under a millisecond to a few ms); watershed is moderate, and K-Means is the slowest.

Source Code

File: experiment6_segmentation.py

```
"""
Experiment 6: Image Segmentation - Thresholding, Otsu, Adaptive, Watershed, K-Means
Install:  pip install opencv-python numpy matplotlib scikit-learn
Usage:    python experiment6_segmentation.py [path_to_image]
Tip: for the watershed demo, use an image of touching objects (coins, cells, rice grains).
"""
import os
import sys
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

os.makedirs("output", exist_ok=True)
rng = np.random.default_rng(3)

# ---------------------------------------------------------------
# Step 1: Load image (fallback: touching dark discs on uneven background)
# ---------------------------------------------------------------
path = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
img = cv2.imread(path)

if img is None:
    print(f"'{path}' not found - generating a synthetic image of touching coins.")
    H, W = 300, 400
    canvas = np.full((H, W), 210, dtype=np.float32)
    centers = [(70, 70, 35), (135, 75, 35), (230, 80, 33), (100, 150, 36),
               (170, 160, 35), (300, 150, 32), (65, 235, 33), (130, 240, 35),
               (255, 235, 33), (325, 230, 28)]
    for (cx, cy, r) in centers:
        cv2.circle(canvas, (cx, cy), r, 70, -1)
    # uneven illumination (gradient) + sensor noise
    canvas *= np.tile(np.linspace(0.5, 1.1, W), (H, 1))
    canvas += rng.normal(0, 8, canvas.shape)
    img = cv2.cvtColor(np.clip(canvas, 0, 255).astype(np.uint8), cv2.COLOR_GRAY2BGR)

# ---------------------------------------------------------------
# Step 2: Grayscale + Gaussian blur
# ---------------------------------------------------------------
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

times = {}

def timed(name, fn):
    t0 = time.perf_counter()
    out = fn()
    times[name] = (time.perf_counter() - t0) * 1000
    return out

# ---------------------------------------------------------------
# Step 3: Global thresholding (fixed T) - objects are DARK, so use THRESH_BINARY_INV
# ---------------------------------------------------------------
T_FIXED = 127
_, th_global = timed("Global", lambda: cv2.threshold(blur, T_FIXED, 255, cv2.THRESH_BINARY_INV))

# ---------------------------------------------------------------
# Step 4: Otsu's thresholding - T chosen automatically from the histogram
# ---------------------------------------------------------------
T_otsu, th_otsu = timed("Otsu", lambda: cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU))
print(f"Fixed T = {T_FIXED} | Otsu T = {T_otsu:.0f}")

# ---------------------------------------------------------------
# Step 5: Adaptive thresholding - local threshold per neighbourhood
# ---------------------------------------------------------------
th_adapt_mean = timed("Adaptive (mean)", lambda: cv2.adaptiveThreshold(
    blur, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY_INV, 101, 10))
th_adapt_gauss = timed("Adaptive (Gaussian)", lambda: cv2.adaptiveThreshold(
    blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 101, 10))

# ---------------------------------------------------------------
# Step 6: Watershed - separates touching objects
# ---------------------------------------------------------------
def watershed_segment(bgr, binary):
    # 'binary' = foreground mask (here: Otsu on the illumination-corrected image)
    kernel = np.ones((3, 3), np.uint8)
    opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)   # remove noise
    sure_bg = cv2.dilate(opening, kernel, iterations=3)                        # certainly background
```

```
    dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)                    # distance to nearest background
    _, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)               # peaks = object centres
    sure_fg = sure_fg.astype(np.uint8)
    unknown = cv2.subtract(sure_bg, sure_fg)                                 # border zone to be resolved
    n_labels, markers = cv2.connectedComponents(sure_fg)
    markers = markers + 1                                                    # background = 1, not 0
    markers[unknown == 255] = 0                                              # 0 = unknown
    markers = cv2.watershed(bgr.copy(), markers)                             # boundaries become -1
    out = bgr.copy()
    out[markers == -1] = (0, 0, 255)                                         # draw boundaries in red
    return out, markers, dist, n_labels - 1

# Illumination correction: estimate the background with a large closing (removes dark objects),
# divide it out, then Otsu works again even though lighting is uneven.
bg_est = cv2.morphologyEx(blur, cv2.MORPH_CLOSE, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (91, 91)))
flat = cv2.normalize(cv2.divide(blur, bg_est, scale=255), None, 0, 255, cv2.NORM_MINMAX)
_, th_flat = cv2.threshold(flat, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
(ws_img, markers, dist, n_objects) = timed("Watershed", lambda: watershed_segment(img, th_flat))
n_adapt_components = cv2.connectedComponents(th_flat)[0] - 1
print(f"Connected components after thresholding only (no watershed): {n_adapt_components} (touching objects merge)")
print(f"Objects found by watershed          : {n_objects}")

# ---------------------------------------------------------------
# Step 7: K-Means colour segmentation
# ---------------------------------------------------------------
def kmeans_segment(bgr, k):
    pixels = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB).reshape(-1, 3).astype(np.float32)
    km = KMeans(n_clusters=k, n_init=5, random_state=0).fit(pixels)
    seg = km.cluster_centers_[km.labels_].reshape(bgr.shape)
    return np.clip(seg, 0, 255).astype(np.uint8), km.labels_.reshape(bgr.shape[:2])

small = cv2.resize(img, None, fx=0.5, fy=0.5)          # speeds up K-Means
km_results = {}
for k in (2, 3, 5):
    km_results[k] = timed(f"K-Means (K={k})", lambda k=k: kmeans_segment(small, k))

# ---------------------------------------------------------------
# Step 8: Quantitative comparison (only possible if ground truth is known)
# ---------------------------------------------------------------
def iou(a, b):
    a, b = a > 0, b > 0
    return (a & b).sum() / max((a | b).sum(), 1)

if not os.path.exists(path):   # synthetic image => we know the true mask
    gt = np.zeros(gray.shape, np.uint8)
    for (cx, cy, r) in centers:
        cv2.circle(gt, (cx, cy), r, 255, -1)
    print("\n---- IoU with ground truth (synthetic image only) ----")
    for name, m in [("Global", th_global), ("Otsu", th_otsu), ("Adaptive (Gaussian)", th_adapt_gauss)]:
        print(f"{name:<22}{iou(m, gt):.3f}")

print("\n---- Execution time (ms) ----")
for name, t in times.items():
    print(f"{name:<22}{t:8.2f}")

# ---------------------------------------------------------------
# Steps 9: Visualise
# ---------------------------------------------------------------
rgb = lambda x: cv2.cvtColor(x, cv2.COLOR_BGR2RGB)
fig, ax = plt.subplots(2, 4, figsize=(16, 8))
panels = [
    ("Original", rgb(img), None), ("Grayscale + blur", blur, "gray"),
    (f"Global T={T_FIXED}", th_global, "gray"), (f"Otsu T={T_otsu:.0f}", th_otsu, "gray"),
    ("Adaptive (Gaussian)", th_adapt_gauss, "gray"), ("Distance transform", dist, "gray"),
    (f"Watershed ({n_objects} objects)", rgb(ws_img), None),
    ("K-Means K=3", rgb(cv2.cvtColor(km_results[3][0], cv2.COLOR_RGB2BGR)), None),
]
for a, (t, im, cm) in zip(ax.ravel(), panels):
    a.imshow(im, cmap=cm); a.set_title(t, fontsize=10); a.axis("off")
plt.tight_layout(); plt.savefig("output/exp6_comparison.png", dpi=150); plt.show()

# Histogram with the Otsu threshold
plt.figure(figsize=(6, 3.5))
plt.hist(blur.ravel(), bins=256, range=(0, 256), color="gray")
plt.axvline(T_otsu, color="red", label=f"Otsu T = {T_otsu:.0f}")
plt.axvline(T_FIXED, color="blue", linestyle="--", label=f"Fixed T = {T_FIXED}")
plt.title("Histogram and thresholds"); plt.legend(); plt.tight_layout()
plt.savefig("output/exp6_histogram.png", dpi=150); plt.show()

# K-Means with different K
fig, ax = plt.subplots(1, 4, figsize=(15, 4))
ax[0].imshow(rgb(small)); ax[0].set_title("Original")
for a, k in zip(ax[1:], (2, 3, 5)):
    a.imshow(km_results[k][0]); a.set_title(f"K-Means K={k}")
for a in ax: a.axis("off")
```

```
plt.tight_layout(); plt.savefig("output/exp6_kmeans.png", dpi=150); plt.show()
```

Questions and Answers

1. What is image segmentation? Why is it considered a fundamental step in computer vision?
Image segmentation is the process of partitioning an image into non-overlapping regions (sets of pixels) that are homogeneous in some property, such as intensity, color, or texture, so that each region corresponds to a meaningful object or part of the scene. It is fundamental because it:

- Isolates objects and regions of interest from the background
- Reduces the amount of data to be analyzed and simplifies the image representation
- Provides the basis for feature extraction, measurement (size, shape, count), and recognition
- Enables scene understanding in medical diagnosis, autonomous driving, and industrial inspection

2. Differentiate between Image Segmentation and Image Classification.

Aspect	Image Segmentation	Image Classification
Goal	Partition the image into regions or label every pixel	Assign one label to the whole image
Output	Mask or label map of the same size as the image	Class label (or class probabilities)
Granularity	Pixel-level, gives location and shape	Image-level, no location information
Example	Outlining a tumor in an MRI scan	Deciding whether an MRI scan shows a tumor or not
Complexity	Higher (dense prediction)	Lower

3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

- **Global thresholding:** a single fixed threshold T is chosen manually; pixels above T become foreground (255) and the rest background (0). Simple and fast, but needs uniform lighting and a suitable T.
- **Otsu's thresholding:** automatically computes the optimal global T by analyzing the histogram and choosing the T that maximizes the between-class variance (minimizes the within-class variance) of the two pixel classes. Works best for bimodal histograms.
- **Adaptive thresholding:** computes a separate threshold for each pixel from its local neighborhood (mean or Gaussian-weighted mean minus a constant C). It copes with uneven illumination and shadows, but is sensitive to block size and C, and can produce noise in flat regions.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?
The watershed algorithm views a grayscale image (or its distance transform) as a topographic relief, where intensity is height. Water floods from marker points (the minima); where floods from different markers meet, 'dams' (watershed lines) are built, which become the boundaries between regions.

- Touching objects form a single connected blob after thresholding, but the distance transform has a separate peak at each object's centre.
- Using those peaks as markers and flooding separates each object along the narrow neck between them.
- In OpenCV: threshold, clean with morphological opening, find sure background (dilation) and sure foreground (distance transform threshold), mark unknown as 0, then call `cv2.watershed`; boundary pixels are labeled -1.
- Limitation: noisy images cause over-segmentation unless markers are used.

5. How is K-Means Clustering applied to image segmentation?

Each pixel is treated as a data point in a feature space, usually its color (R, G, B), optionally with position (x, y). The image is reshaped to an N x 3 array, and K-Means with a chosen K is run:

1. Initialize K cluster centres
2. Assign every pixel to the nearest centre (Euclidean distance)
3. Recompute each centre as the mean of its assigned pixels
4. Repeat until the centres stop changing

Each pixel is then replaced by the colour of its centre (or given its cluster label), producing K segments. K determines the number of regions, and the method is unsupervised.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Aspect	Threshold-based	Clustering-based (K-Means)
Principle	Compares pixel intensity with one or more thresholds	Groups pixels by feature similarity (color, intensity, position)
Input	Usually grayscale	Color or multi-channel features
Parameters	Threshold, block size	Number of clusters K, initialization
Speed	Very fast	Slower (iterative)
Strengths	Simple, efficient, good for high-contrast objects	Handles multiple classes and color images; no manual threshold
Weaknesses	Fails with uneven lighting or overlapping intensities; ignores spatial information	Needs K; ignores spatial connectivity; sensitive to initialization and noise

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

- Uneven lighting, shadows, and glare make a single threshold fail
- Background texture or clutter has intensity and color similar to the objects
- Low contrast and blurred or weak boundaries between object and background
- Noise causing over-segmentation or false regions
- Touching or overlapping objects that merge into one region
- Variation in object size, shape, and color, and the need to choose parameters (K, T, block size) for each image
- Remedies: illumination correction, adaptive methods, morphological operations, marker-based watershed, and deep learning models

8. Mention five real-world applications where image segmentation plays a critical role.

1. Medical imaging: tumor, organ, and cell segmentation in MRI, CT, and microscopy
2. Autonomous driving: road, lane, vehicle, and pedestrian segmentation
3. Satellite and remote sensing: land-use classification, crop and water body mapping
4. Industrial inspection: defect detection and part counting
5. Document analysis and OCR (text and background separation); also video surveillance, agriculture, and background removal in photo editing

9. How does image segmentation improve the performance of object detection and image recognition systems?

Segmentation separates objects from background clutter so later stages work only on relevant regions. This reduces computation and false detections, gives precise boundaries and shapes for measurement and feature extraction, makes features cleaner (since they are computed on the object alone), and helps count and localize individual objects. As a result, classifiers and detectors become more accurate, faster, and more robust to complex backgrounds.

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Aspect	Traditional (threshold, watershed, K-Means)	Deep learning (U-Net, Mask R-CNN)
Approach	Hand-designed rules using intensity, color, edges	Learned features from large labeled datasets
Data needed	None (unsupervised or rule-based)	Large annotated datasets
Accuracy	Good for simple, controlled scenes; poor for complex scenes	State-of-the-art on complex, cluttered, and varied scenes
Semantic understanding	None; regions have no class meaning	Yes: semantic (U-Net) and instance segmentation (Mask R-CNN gives a mask per object)
Computation	Light, runs on CPU in real time	Heavy; needs GPU for training and often inference
Interpretability	Transparent and easy to tune	Black box, harder to interpret
Typical use	Preprocessing, simple industrial tasks, quick prototypes	Medical imaging (U-Net), autonomous driving, object instance segmentation (Mask R-CNN)

Conclusion

Global, Otsu, and adaptive thresholding, watershed, and K-Means segmentation were implemented using Python and OpenCV and compared. Global and Otsu thresholding are fast but need uniform lighting, adaptive thresholding handles uneven illumination, watershed separates touching objects, and K-Means segments by color. The best technique depends on the image: lighting conditions, whether objects touch, and whether color or intensity separates the regions. For complex scenes, deep learning methods such as U-Net and Mask R-CNN give better results at greater computational cost.